

question 1

We want to evaluate the amount of messages in the k^{th} step:

- We will denote two adjacent active neighbors in the k^{th} step as P_i, P_{i+1}
- We will denote the distance between them, d_i as the number of relay processes between them in the k^{th} step.
- We notice that the sum of such distances amounts to $2N$, the network is one-directional and there is only one overlap in the distance intervals :
$$\sum_{i=1}^M (d_i + d_{i+1}) = \sum_{i=1}^M d_i + \sum_{i=1}^M d_{i+1} = N + N = 2N$$
- where M is the number of active processes in the k^{th} step
- Therefore, the amount of messages per steps is independent of the number of active processes in said step.

We want to evaluate the number of such steps asymptotically:

- Let us define 3 arbitrary processes $X \rightarrow Y \rightarrow Z$.
For Z to remain active in the next step, the next condition must apply :
$$ID(Y) > ID(X) \wedge ID(Y) > ID(Z)$$
- Let W be the next active process after Z . For W to remain active in the next step, the next condition must apply :
$$ID(Z) > ID(Y) \wedge ID(Z) > ID(W)$$
- Notice that if both Z, W remain active, it will result in a contradiction meaning no 2 adjacent neighbors can remain active.

Therefore, at most half of the active processes will remain active in the next step and so the total number of steps is : $\log_2 N$

Total messages complexity

- Total messages = #steps $\times$ #messages\step = $2N \times \log_2 N$
- Messages complexity : $\mathcal{O}(N \log N)$

question 2

We will examine each property separately and decide if it is necessary for keeping the $\Omega(N \log N)$ lower bound.

property 1 - Valid

We see that changing the algorithm from electing the maximum to , for say, electing the minimum ID is symmetrically equivalent and so the message complexity does not change. Therefore, this specific property is not essential for the $\Omega(N \log N)$ lower bound.

property 2 - Valid

We will prove by contradiction.

Let us denote an algorithm A - that elects a leader without notifying the rest of the processes. Lets us assume by way of contradiction that A works with a message complexity $o(N \log N) < \Omega(N \log N)$.

Now, let us construct a new algorithm A' that fulfills all original requirements:

1. run the A algorithm to elect a leader - $o(N \log N)$
2. notify the rest of the processes - N

The time complexity of A' is : $o(N \log N) + N = o(N \log N)$

The contradiction :

We designed an algorithm that meets all 3 properties in $o(N \log N) < \Omega(N \log N)$ in contradiction to the original claim.

property 3 - Invalid

In lecture 1 we talked about the `time slice` algorithm which works in message complexity $\mathcal{O}(N)$ which is faster than the lower bound : $\mathcal{O}(N) < \Omega(N \log N)$.

question 3

The algorithm :

1. Init. - each process P_i holds the number : $x = UID_i$
2. We use `time slice` algorithm to elect the minimum UID as the leader.
 - Because the minimum UID in the ring is ≤ 2 , the process with the minimum UID will wake up first (at round 1 or 2) and become the leader.
3. The leader initiates a message containing its value x and sends it to the next process.
4. On receiving a message - the process performs the logical xor operation of the stored value with the received message : $x \leftarrow x \oplus UID_i$ and sends it forward.
5. When the leader receives the message (after exactly N rounds)
 - The message contains the xor operation of all used $UIDs$ (without the ghost UID)
 - The leader then performs the logical xor operation between said message and all values $\{1 \dots N + 1\}$.

- By the property of the logical xor operation, we will be left with the ghost ID .

6. The leader shares the ghost ID with the rest of the class : $Ghost\ ID = M \oplus \bigoplus_{k=1}^{N+1} k$

Message complexity : exactly 2 messages make a full traversal of the ring. The accumulation message takes N hops and the broadcast message takes N hops for a total of :

$N + N = 2N = \mathcal{O}(N)$ messages

Round complexity :

- The `time slice` algorithm takes maximum of $N + 1$ rounds (since $N + 1$ is the maximum possible ID)
- The accumulation and broadcast phases take N rounds each.
- **Total** $\leq (N + 1) + N + N = 3N + 1 = \mathcal{O}(N)$ Rounds.

Space complexity : $\mathcal{O}(1)$ memory complexity and the messages only ever contain a single integer.

question 4

A graph is complete if all nodes have the exact same degree Δ , and the total number of nodes is exactly $\Delta + 1$.

The algorithm

1. Init - the initiator v_0 counts it's local edges. Let this be Δ .
2. v_0 sends a message containing it's degree : $WAKE(\Delta)$ to all it's neighbors simultaneously.
3. when an inactive node receives $WAKE(\Delta)$ for the first time from a neighbor (which becomes its parent) : $\langle k, p \rangle$
 - it checks its own degree. If $|N(v_i)| \neq \Delta$ it marks a local variable `is_complete = False` otherwise `is_complete == True` .
 - It then forwards $WAKE(\Delta)$ to all its neighbors.
4. **Convergecast Phase** : Once a node v_i has received messages from all its edges, it replies to its parent with a message : $echo(count, complete)$
 1. `count` : the total sum of nodes in its subtree (including itself).
 2. `complete` : True only if its own `is_complete == True` and all its children returned `True` .
5. **Decision Phase** :
 - Eventually the initiator, v_0 receives `echo` messages from all its neighbors.
 - It sums the `count` from all of its neighbors +1 (for itself) to get the `total_nodes` .
 - v_0 returns `True` if `total_nodes ==  $\Delta + 1$` and all neighbors return `True` otherwise it returns `False` .

Time complexity : The broadcast traverses outwards reaching the furthest node in exactly d steps. The `echo` traverses back in at most d steps. Total time is : $2d = \mathcal{O}(d)$.

Message complexity : every edge in the network is traversed exactly twice (once by a `wake` message and once by an `echo` message). total messages $2|E| = \mathcal{O}(|E|)$.

Message size : The messages only carry integers up to the maximum degree or maximum node count which are bounded by N requiring $\mathcal{O}(\log N)$ bits.

question 5

The algorithm

1. Initialization :

- all distances from the leader L are initialized as follows :

$$\text{dist}(L, v) = \begin{cases} 0 & : v = L \\ \infty & : v \neq L \end{cases}$$

- All parents $\text{parent}(\text{UID}_p)$ of all processes are initialized to `Null`.
- Every process u has 2 variables : `my_dist` , `parent`.

2. upon u receiving a message from process v - `from(d)` :

- If $d + 1 < \text{my_dist}$:
 - $\text{my_dist} \leftarrow d + 1$
 - $\text{parent} \leftarrow v$
 - process u sends a `from(my_dist)` message to all of its neighbors
 - process u sends a `to(UID(u), my_dist)` message to its parent

3. upon u receiving a message from process v - `to(UID(v), d)` :

- If $u == L$:
 - L locally records that process `UID(v)` is at distance d
- Else :
 - process u simply forwards the `to(UID(v), d)` message to its parent.

Correctness :

The algorithm relies on the *bellman-ford* distance principal of finding shortest distances. Upon some process u receiving a better distance it sends an update message to the leader. Thus the algorithm finds the shortest path tree.

Time complexity :

In the worst case : asynchronous scenario, it can take $\mathcal{O}(|V| \cdot |E|)$ time for all shortest paths to stabilize.

Message complexity :

- Downward (`from`) : in the worst case (asynchronous) processes might update their distances multiple times leading $\mathcal{O}(|V| \cdot |E|)$ messages.
- Upwards (`to`) : every time a process updates its distance it sends a message that travels at most the diameter of the network $\implies \mathcal{O}(|V| \cdot D)$.

⚡ Dog seems very worried!

